

Grupo Cordillera

Informe tecnico de arquitectura del proyecto final

Este informe resume el proyecto con foco en arquitectura, capas, tecnologias, patrones, comunicacion, pruebas, despliegue y cumplimiento de requisitos de entrega.

1. Vision general

Grupo Cordillera implementa un dashboard empresarial de KPIs con una arquitectura distribuida. El navegador consume un frontend React/TypeScript; las rutas API pasan por un API Gateway Nginx; el BFF Node/Express adapta el contrato para la UI; y los dominios se separan en microservicios Spring Boot.

- Separacion estricta de responsabilidades: presentacion, gateway, BFF, negocio y datos.
- Comunicacion sincrona HTTP/REST con JSON.
- Persistencia aislada por servicio para usuarios, KPIs y reportes.
- Despliegue local por Docker Compose y despliegue de presentacion por Kubernetes.

2. Componentes principales

Componente	Ruta	Tecnologia	Responsabilidad
Frontend	frontend/	React, TypeScript, Vite, Tailwind	Interfaz de dashboard, KPIs, alertas y creacion de usuarios.
API Gateway	backend/api-gateway/	Nginx	Entrada HTTP y proxy de /api/* hacia BFF.
BFF	backend/bff/	Node.js, Express	Proxy, fachada y agregador de dashboard.
ms-auth	backend/ms-auth/	Spring Boot, JWT	Login, registro y token RS256.
ms-user	backend/ms-user/	Spring Boot, JPA, Flyway	Gestion y autenticacion de usuarios.
ms-kpis	backend/ms-kpis/	Spring Boot, JDBC, Flyway	Consultas de indicadores del dashboard.
ms-report	backend/ms-report/	Spring Boot, JPA, Flyway	Gestion de reportes.

3. Comunicacion entre capas

Flujo nominal: Browser -> Frontend -> API Gateway -> BFF -> Microservicio -> PostgreSQL. El BFF expone /api/auth, /api/kpis, /api/users y /api/reports. Para /api/dashboard consulta varios endpoints de KPIs en paralelo y retorna una respuesta compuesta.

4. Patrones de arquitectura y diseno

- Microservices by domain: auth, user, kpis y report.
- Database per Service: user_db, kpis_db y report_db; auth se mantiene stateless y delega usuarios a ms-user.
- Layered Architecture: controller, service, repository, dto/model.
- Backend for Frontend: contrato optimizado para la SPA.

- Strategy + Factory en ms-kpis para resolver cada tipo de KPI mediante KpiType.
- Global Exception Handler en cada microservicio Java para respuestas coherentes.

5. Datos e infraestructura

Los servicios con persistencia usan PostgreSQL 16 y migraciones Flyway versionadas. Docker Compose levanta bases, servicios, BFF, gateway y frontend con health checks. Kubernetes define namespace, deployments, services, configmaps, secrets, bases de datos e ingress para grupo-cordillera.local.

6. Calidad y pruebas

Area	Herramienta	Criterio
Java services	JUnit + JaCoCo	mvn verify genera reporte y exige minimo 60% line coverage.
BFF	Jest + Supertest	npm test ejecuta pruebas HTTP y cobertura minima 60%.
Frontend	TypeScript + ESLint	npm run build y npm run lint validan tipos y convenciones.

7. Cumplimiento relevante

- Repositorio GitHub y rama de trabajo basada en cambios revisables por Pull Request.
- Estructura documentada segun /backend, /frontend y /docs.
- Swagger disponible en cada microservicio Spring Boot.
- README tecnico por componente backend y frontend.
- Documentos finales en docs/: report.pdf, presentation.pdf y caso-estudio.pdf.